SQL Explanation Grader: baseline vs RAG

Experiment **rag_vs_baseline_dev** · split **dev** (50 items) · generator **llama3.2:3b** · judge **qwen2.5:3b** (legacy_schema) · generated 2026-10-26 18:07

How to read this report. "Judge approval" is the share of explanations an LLM judge graded good. It is an automated, unverified measurement, not ground truth. Deterministic checks are rule-based heuristics computed from the SQL structure. Agreement of the judge with reference labels is reported separately (results/rag/judge_agreement.json).

Headline comparison

Metric	Baseline (prompt only)	RAG
Judge approval (95% CI)	30.0% (15/50) [19-44]	22.0% (11/50) [13-35]
Approval, abstentions count as fail	30.0% (15/50) [19-44]	22.0% (11/50) [13-35]
Rubric pass (judge's own scores)	n/a	n/a
Mean correctness (1-5)	n/a	n/a
Mean completeness (1-5)	n/a	n/a
Mean clarity (1-5)	n/a	n/a
Mean groundedness (1-5)	n/a	n/a
Explanations with unsupported claims (judge)	n/a	n/a
Deterministic check coverage	0.96	0.98
Invented numbers (rule-based)	10.0% (5/50) [4-21]	12.0% (6/50) [6-24]
Raw SQL keywords in text	10.0% (5/50) [4-21]	66.0% (33/50) [52-78]
Abstentions	0.0% (0/50) [0-7]	0.0% (0/50) [0-7]
Generation errors	0	0
Mean end-to-end latency (s)	6.6	17.5
Mean retrieval time (s)	0.00	1.49
Mean input tokens	571	2647
Mean output tokens	104	266

AI-assisted reference labels (not human): share labelled good

	Baseline	RAG
Labelled good	20/50	10/50
Paired: only baseline good / only RAG good	16	6
RAG minus baseline (95% bootstrap CI)	-20.0 pts (-38.0 to -2.0)	McNemar p = 0.052

Labels and their decision rules: data/labels/. They are not produced by the judge in this report.

Paired comparison (same items, both judged)

50 pairs. Both good: 6; both bad: 30; only baseline good: 9; only RAG good: 5. Approval difference RAG minus baseline: -8.0 points (95% bootstrap CI -22.0 to +6.0). Exact McNemar p = 0.424.

Grounding and retrieval (RAG condition)

Metric	Value
Answers with at least one citation	90.0% (45/50) [79-96]
Citation validity (cited source was provided)	1.000
Answers citing a source that was not provided	0.0% (0/50) [0-7]
Share of sentences carrying a citation	0.447

Metric	Value
Lexical support of cited sentences (heuristic)	0.806
Model said evidence was insufficient	0
Retrieval feature coverage	1.000
Evidence judged sufficient before generation	100.0% (50/50) [93-100]
Grounding status counts	{"grounded": 45, "weakly_grounded": 5}

Retrieval evaluation (no LLM involved)

Project-authored by Claude (AI-assisted) on 2026-10-26 by reading the section headings of each corpus document. Not human-reviewed yet. Treat retrieval scores computed with it as rule-based estimates, not human relevance judgements.

Chunk	Strategy	Mode	MRR	Recall@5	Prec@5	nDCG@5	Ctx recall	ms/query
400	decomposed	dense	0.824	0.925	0.556	0.616	0.892	161
400	decomposed	bm25	0.642	0.836	0.568	0.560	0.960	5
400	decomposed	hybrid	0.690	0.867	0.585	0.590	0.893	162
400	probe	hybrid	0.564	0.596	0.397	0.417	0.711	50
400	raw_sql	hybrid	0.691	0.797	0.522	0.538	0.799	58
200	decomposed	dense	0.784	0.899	0.571	0.584	0.943	161
200	decomposed	bm25	0.710	0.837	0.637	0.627	0.963	7
200	decomposed	hybrid	0.640	0.878	0.604	0.569	0.948	163
300	decomposed	dense	0.764	0.676	0.511	0.548	0.934	162
300	decomposed	bm25	0.658	0.842	0.597	0.576	0.962	5
300	decomposed	hybrid	0.644	0.834	0.535	0.531	0.940	160
500	decomposed	dense	0.694	0.727	0.448	0.500	0.873	160
500	decomposed	bm25	0.728	0.839	0.589	0.604	0.968	4
500	decomposed	hybrid	0.647	0.854	0.566	0.564	0.874	166
600	decomposed	dense	0.743	0.645	0.458	0.532	0.932	162
600	decomposed	bm25	0.705	0.835	0.548	0.571	0.913	6
600	decomposed	hybrid	0.615	0.830	0.490	0.502	0.899	160

Breakdown by SQL feature (dataset label)

Feature	n	Baseline approval	RAG approval	Base checks	RAG checks
GROUP BY with HAVING	8	25.0% (2/8) [7-59]	37.5% (3/8) [14-69]	0.88	0.88
INNER JOIN of two tables	9	66.7% (6/9) [35-88]	11.1% (1/9) [2-44]	1.00	1.00
JOIN of three tables with ORDER BY and LIMIT	8	37.5% (3/8) [14-69]	25.0% (2/8) [7-59]	0.88	1.00
LEFT JOIN with IS NULL filter	3	100.0% (3/3) [44-100]	66.7% (2/3) [21-94]	1.00	1.00
simple SELECT with WHERE	5	0.0% (0/5) [0-43]	0.0% (0/5) [0-43]	1.00	1.00
subquery in WHERE	10	10.0% (1/10) [2-40]	30.0% (3/10) [11-60]	1.00	1.00
window function such as ROW_NUMBER or RANK	7	0.0% (0/7) [0-35]	0.0% (0/7) [0-35]	1.00	1.00

Items where the two conditions were graded differently

q006 · LEFT JOIN with IS NULL filter

```
SELECT * FROM books LEFT JOIN loans ON books.id = loans.book_id WHERE loans.returned IS NULL;
```

Baseline: good

This query retrieves all books that have never been loaned out. It starts by selecting all columns from the books table. Then, it joins the loans table to the books table on the condition that the book's id matches the loans table's book_id. The LEFT JOIN ensures that all books are included in the result, even if there is no matching loan record. The WHERE clause then filters the results to only include rows where the returned status is NULL, meaning the book has not been returned yet.

Judge: The explanation correctly describes the query's operation, including the use of LEFT JOIN and the WHERE clause to filter out returned books. It covers all aspects of the query.

RAG: bad

This query retrieves all books that have not been loaned out, along with their title, author, and year. The query starts by selecting all columns from the books table, and then joins the loans table to the books table on the condition that the book has not been returned (i.e., the returned column is NULL). This is a left outer join, which means that all books will be included in the results, even if they have not been loaned out. The extra row for each book that has not been loaned out contains NULL values for the columns that would normally contain values from the loans table. [S1] [S2]

Judge: It incorrectly states that the query retrieves all books that have not been loaned out. The query actually retrieves books that have not been returned, which is different from not being loaned out. It also fails to cover the WHERE clause correctly.

q029 · INNER JOIN of two tables

```
SELECT patients.id, doctors.id, appointments.appt_date, patients.name AS patient_name, doctors.name AS doctor_name FROM patients INNER JOIN appointments ON patients.id = appointments.patient_id INNER JOIN doctors ON appointments.doctor_id = doctors.id;
```

Baseline: good
This query retrieves a list of patients and their corresponding appointments, along with the doctor's name and appointment date. The query starts by selecting the patient's ID and the doctor's ID, as well as the appointment date. It then joins the patients table with the appointments table on the patient's ID, and the appointments table with the doctors table on the doctor's ID. The result is a list of patients with their corresponding appointments, including the doctor's name and appointment date. The query does not include any filters or grouping, and it returns all rows from the joined tables. The result set includes the patient's ID, the doctor's ID, the appointment date, the patient's name, and the doctor's name, with the patient's name and doctor's name aliased as patient_name and doctor_name, respectively.
Judge: The explanation correctly describes the query's purpose, joins, and columns selected. It also clarifies the aliasing used.

RAG: bad
This query returns the patient's ID, doctor's ID, appointment date, patient's name, and doctor's name. The query joins the patients table with the appointments table on the patient's ID, and then joins the appointments table with the doctors table on the doctor's ID. The result is the cartesian product of the patients and doctors datasets, with the appointment date and names added. [S1] The query uses the standard join syntax, but SQLite allows for non-standard join types, such as 'OUTER LEFT NATURAL JOIN'. [S5] The query does not use any non-standard join types. [S2] The query uses the 'INNER JOIN' syntax, which is allowed in SQLite. [S3] The query uses the cartesian product of the patients and doctors datasets, which is the default behavior for joins in SQLite. [S4] The query uses the 'WHERE' clause to filter out rows with NULL values, which is the standard behavior in SQLite. [S6] The query does not use any 'FULL JOIN' or 'FULL OUTER JOIN' syntax, which is allowed in SQLite. [S3] The query uses the cartesian product of the patients and doctors datasets, which is the default behavior for joins in SQLite. [S4] The query uses the 'WHERE' clause to filter out rows with NULL values, which is the standard behavior in SQLite. [S5] The query uses the standard join syntax, but SQLite allows for non-standard join types, such as 'OUTER LEFT NATURAL JOIN'. [S1] The query uses the standard join syntax, but SQLite allows for non-standard join types, such as 'OUTER LEFT NATURAL JOIN'. [S2] The query uses the standard join syntax, which is allowed in SQLite. [S3] The query uses the cartesian product of the patients and doctors datasets, which is the default behavior for joins in SQLite. [S4] The query uses the 'WHERE' clause to filter out rows with NULL values, which is the standard behavior in SQLite. [S6] The query does not use any 'FULL JOIN' or 'FULL OUTER JOIN' syntax, which is allowed in SQLite.
Judge: The explanation incorrectly states that the query uses the cartesian product of the patients and doctors datasets, which is not the case. The query uses INNER JOIN, not a cartesian product.

---

q043 · subquery in WHERE
```
SELECT * FROM loans WHERE book_id IN (SELECT id FROM books WHERE year > 2000);
```

Baseline: good
This query retrieves all rows from the loans table where the book_id matches a book that was published after 2000. In other words, it shows all loans of books published in the 21st century. The subquery first identifies the book_ids of books published in the 21st century, and then the main query selects the corresponding loan records.
Judge: The explanation correctly describes the query's operation, covers all parts of the query, and uses clear language.

RAG: bad
This query selects all rows from the loans table where the book_id is in the list of book_ids from the books table that were published after 2000. The IN operator is used to compare the book_id in the loans table to the list of book_ids in the books table. The subquery on the right-hand side of the IN operator must return exactly one column, which is the book_id. The result of the IN operator is true if any of the book_ids in the loans table match the book_id in the books table, and false otherwise. [S2]
Judge: The explanation does not cover the subquery and the IN operator correctly. It should explain that the subquery returns multiple book_ids, and the IN operator checks if each book_id in loans matches any of the book_ids in the subquery.

---

q074 · subquery in WHERE
```
SELECT name FROM patients WHERE birth_year = (SELECT birth_year FROM patients WHERE id = 1);
```

Baseline: bad
This query selects the names of patients from the patients table. The query uses a subquery to find the birth year of the patient with id 1, and then filters the results to only include patients with that birth year. The result is a list of names of patients born in the same year as the patient with id 1.
Judge: The explanation does not cover the subquery and the filtering logic correctly. It incorrectly states the query selects names of patients from the patients table, but the query actually uses a subquery to find a specific patient's birth year and filters patients by that year.

RAG: good
The query selects the name of the patient born in the same year as the patient with id 1. This is done by using a subquery to find the birth year of the patient with id 1, and then using this value to filter the patients. The subquery is correlated because it references the outer query's column (id 1). The result is a list of patient names born in the same year as the patient with id 1. [S2]
Judge: The explanation correctly identifies the query's operation, covers the subquery, and explains the correlation.

---

q082 · subquery in WHERE
```
SELECT students.name, courses.title FROM students JOIN enrollments ON students.id =
enrollments.student_id JOIN courses ON enrollments.course_id = courses.id WHERE courses.credits =
```

```
(SELECT MAX(credits) FROM courses);
```

Baseline: bad
This query selects the names of students and the titles of courses they are enrolled in. It starts by joining the students table with the enrollments table, where each student is matched with the courses they are enrolled in, and then joins the enrollments table with the courses table. The query then filters the results to only include courses with the highest number of credits. The result is a list of students and the courses they are enrolled in, with only the courses having the most credits.
Judge: The explanation does not cover the subquery used to filter courses by the maximum number of credits.

RAG: good
This query returns the names of students enrolled in the course with the most credits. It joins the students table with the enrollments table on the student ID, and then joins the enrollments table with the courses table on the course ID. The WHERE clause filters the courses to only include the one with the most credits, which is determined by a subquery that finds the maximum credits in the courses table. The result is a list of student names, one for each course with the most credits.
Judge: The explanation correctly describes the query's operations, including the joins, filters, and aggregations. It also clearly explains the subquery used to find the maximum credits.

---

q083 · subquery in WHERE

```
SELECT students.name, courses.title FROM students JOIN enrollments ON students.id =
enrollments.student_id JOIN courses ON enrollments.course_id = courses.id WHERE courses.credits >
(SELECT AVG(credits) FROM courses);
```

Baseline: bad
This query selects the names of students and the titles of courses they are enrolled in. It joins the students, enrollments, and courses tables to find the courses each student is taking. The query then filters the results to only include courses that have more credits than the average credits of all courses. In other words, it shows students who are taking courses that are significantly more challenging than the average course.
Judge: The explanation does not cover the WHERE clause correctly. It should state that the query filters courses by having more credits than the average, not that it shows students taking significantly more challenging courses.

RAG: good
This query returns the names of students enrolled in courses with more credits than the average credits of all courses. The query first joins the students table with the enrollments table on the student ID, and then joins the enrollments table with the courses table on the course ID. The WHERE clause filters the results to include only courses with more credits than the average credits of all courses. The average credits are calculated using a subquery. The result is the names of students enrolled in courses with more credits than the average credits of all courses.
Judge: The explanation covers every aspect of the query including joins, filters, grouping, and aggregate. It also correctly explains the subquery and the final result.

---

q089 · GROUP BY with HAVING

```
SELECT course_id, COUNT(*) FROM enrollments GROUP BY course_id HAVING COUNT(*) % 2 = 0;
```

Baseline: bad
This query selects the course IDs of students who have enrolled in an even number of courses. It does this by counting the number of courses each student is enrolled in, grouping the results by course ID, and then filtering the groups to only include those with an even count of courses. The exact course IDs returned are the ones that meet this condition, which is why the query uses the `COUNT(*)` function to count the number of courses each student is enrolled in.
Judge: The explanation does not cover the GROUP BY and HAVING clauses, which are necessary for the query to work as intended.

RAG: good
The query selects the course_id of courses that have an even number of enrollments. The COUNT(*) expression is evaluated for each group of rows, and the HAVING clause filters out groups where the count is odd. [S1] [S2]
Judge: The explanation correctly describes the query's operation, including the use of COUNT(*) and HAVING to filter groups.

---

q092 · INNER JOIN of two tables

```
SELECT customers.name, orders.quantity, orders.order_date FROM customers INNER JOIN orders ON
customers.id = orders.customer_id;
```

Baseline: good
This query selects the customer's name, the quantity of products ordered, and the date of the order from the customers and orders tables. It joins the two tables based on the matching customer ID, and only includes rows where the customer ID exists in both tables. The result set will contain the customer's name, the quantity of products ordered, and the date of the order for each customer who has placed an order.
Judge: The explanation correctly describes the query's operation, covers all aspects (join, filters, grouping, aggregate, ordering), and uses clear language.

RAG: bad
This query returns the customer's name, the quantity of the order, and the order date. The query joins the customers table with the orders table on the customer's ID, and only includes rows where the quantity is not NULL. [S3] The extra NULL rows for non-matched rows on the other operand are added after ON clause processing but before WHERE clause processing, so the ON clause does not affect the inclusion of NULL rows. [S3] The query uses the INNER JOIN operator, which is equivalent to a comma-join or CROSS JOIN without an ON clause. [S2] The result is the cartesian product of the customers and orders tables, excluding rows where the quantity is NULL. [S4]
Judge: Incorrect explanation of the query behavior and missing details

---

q096 · INNER JOIN of two tables

```
SELECT orders.id, customers.name, products.name, orders.quantity, orders.order_date FROM orders INNER
JOIN customers ON orders.customer_id = customers.id INNER JOIN products ON orders.product_id =
products.id;
```

Baseline: good
This query retrieves information from three tables: customers, products, and orders. It selects specific columns from each table: the order ID, customer name, product name, order quantity, and order date. The query joins the orders table with the customers table on the customer ID, and then joins the result with the products table on the product ID. The result set includes all rows from the orders table, along with the matching customer and product information. The query does not filter or limit the number of rows returned, so it will include all orders, customers, and products that are associated with each order.
Judge: The explanation covers the join conditions, selected columns, and the overall structure of the query correctly. It also mentions that the query does not filter or limit the number of rows, which is accurate.

RAG: bad
This query returns all columns from the orders table, along with the customer's name and product's name, and the quantity and order date of each order. The query joins the orders table with the customers table on the customer's id, and the products table on the product's id. The result is the cartesian product of the left and right-hand datasets, with no filtering or aggregation. [S2] SQLite is not nearly so fussy about join syntax as standard SQL, and allows the use of an ON or USING clause on a comma-join or CROSS JOIN, and will omit the ON or USING clause from any join at all. [S3] All joins in SQLite are based on the cartesian product of the left and right-hand datasets, and the result is simply the cartesian product of the left and right-hand datasets if there is no ON or USING clause. [S4] If a WHERE clause is specified, the WHERE expression is evaluated for each row in the input data as a boolean expression, and only rows for which the WHERE clause expression evaluates to true are included from the dataset. [S4] In this case, there is no WHERE clause, so all rows from the orders table are included. [S5] The query uses the standard JOIN syntax, but allows for non-standard join types such as OUTER LEFT NATURAL JOIN, which is equivalent to NATURAL LEFT OUTER JOIN. [S6] The result of the query includes extra rows with NULL values in the columns that would normally contain values copied from the left-hand input dataset.
Judge: The explanation does not cover every aspect of the query, including the WHERE clause and the result set. It also incorrectly states that the query includes extra rows with NULL values, which is not the case. The query does not use a cartesian product but an INNER JOIN, and it does not include all columns from the orders table as stated.

---

q097 · JOIN of three tables with ORDER BY and LIMIT
```
SELECT patients.name, doctors.name, appointments.appt_date FROM patients JOIN appointments ON patients.id = appointments.patient_id JOIN doctors ON appointments.doctor_id = doctors.id ORDER BY appointments.appt_date LIMIT 1;
```

Baseline: good
This SQL query retrieves the names of patients, doctors, and appointment dates from the database. It starts by joining the patients, appointments, and doctors tables based on their matching IDs. The query then orders the results by the appointment date in ascending order, so the earliest appointment date appears first. Finally, it limits the results to a single row, effectively showing the earliest appointment date and the corresponding patient and doctor names.
Judge: The explanation correctly describes the query's operations, covers all aspects of the query (joins, filters, grouping, aggregate, ordering, and LIMIT), and uses clear language.

RAG: bad
This query returns the names of patients, doctors, and the date of their appointments, sorted by appointment date, and limited to one row. The query joins the patients, appointments, and doctors tables based on their IDs, and filters out rows where the appointment date is not specified. The result is ordered by appointment date, with earlier dates appearing first, and NULL values are considered smaller than any other values. The query uses a LIMIT clause to return only one row, and an OFFSET clause to skip the first row. [S3] [S5]
Judge: The explanation incorrectly states that the query uses an OFFSET clause, which is not present in the original query. The query uses LIMIT instead.

---

q101 · JOIN of three tables with ORDER BY and LIMIT
```
SELECT patients.name, doctors.name, appointments.appt_date, appointments.fee FROM patients JOIN appointments ON patients.id = appointments.patient_id JOIN doctors ON appointments.doctor_id = doctors.id ORDER BY patients.name LIMIT 5;
```

Baseline: bad
This query retrieves the names of patients, doctors, appointment dates, and fees from the database. It joins three tables: patients, appointments, and doctors. The query starts by matching patients with their corresponding appointments based on the patient's ID, and then matches appointments with doctors based on the doctor's ID. The results are ordered alphabetically by patient name, and only the first 5 rows are returned. The query produces the names of patients, doctors, appointment dates, and fees, with no additional calculations or aggregations. The appointment dates are returned as strings, and the fees are returned as real numbers.
Judge: The explanation does not cover the LIMIT 5 clause, which is part of the query.

RAG: good
This query returns the patient's name, doctor's name, appointment date, and fee for the 5 most recent appointments, sorted by patient's name. The query joins the patients, doctors, and appointments tables based on their IDs, and then limits the result to the 5 most recent appointments. The appointment date is used to determine the order of the results, and the fee is included as part of the result. [S3]
Judge: The explanation correctly describes the query's operation, covers all aspects of the query (joins, filters, grouping, aggregate, ordering, and LIMIT), and uses clear language.

---

q109 · JOIN of three tables with ORDER BY and LIMIT
```
SELECT patients.name, doctors.name, appointments.appt_date, appointments.fee, appointments.id FROM
```

```
patients JOIN appointments ON patients.id = appointments.patient_id JOIN doctors ON
appointments.doctor_id = doctors.id ORDER BY appointments.appt_date LIMIT 10;
```

Baseline: good

This SQL query retrieves information from three tables: patients, doctors, and appointments. It starts by joining the patients table with the appointments table on the patient_id column, and then joins the doctors table with the appointments table on the doctor_id column. This means that for each appointment, it finds the corresponding patient and doctor. The query then selects specific columns from these tables: the patient's name, the doctor's name, the appointment date, the fee, and the appointment ID. The results are ordered by the appointment date, so the earliest appointments come first, and the query limits the output to the 10 most recent appointments. The result is a list of the 10 most recent appointments, including the patient's name, the doctor's name, the appointment date, the fee, and the appointment ID.

Judge: The explanation correctly describes the query's operations, including joins, filters, grouping, aggregate, ordering, and LIMIT. It uses clear language and covers all aspects of the query.

RAG: bad

This query returns the patient's name, doctor's name, appointment date, fee, and appointment ID from the patients, doctors, and appointments tables, joined on the patient ID and doctor ID columns. The results are sorted by appointment date in ascending order, and the first 10 rows are returned. [S3] The ORDER BY clause determines the order in which rows are returned, and [S6] the LIMIT clause limits the number of rows returned. [S2] The JOIN syntax is flexible in SQLite, allowing for comma-joins, CROSS JOINs, and NATURAL joins without an ON or USING clause. [S1] The CROSS JOIN operator is not used in this query, but it would produce the same result as an INNER JOIN or JOIN operator. [S5] The WHERE clause is not used in this query, but it would filter the results based on a condition. [S4] The WITH clause is not used in this query, but it would allow for recursive queries. [S6] The OFFSET clause is not used in this query, but it would allow for skipping a specified number of rows at the beginning of the result set. [S3] The ORDER BY clause applies to the entire compound SELECT statement, not just the final SELECT. [S6] The LIMIT clause applies to the entire compound SELECT statement, not just the final SELECT. [S6] The LIMIT clause can be used with the OFFSET clause to limit the number of rows returned and skip a specified number of rows at the beginning of the result set.

Judge: The explanation does not cover every aspect of the query, such as the WHERE clause and the use of LIMIT with OFFSET.